

SILICON PHOTONICS DESIGN & AUTOMATION

Revised Industry-Aligned Curriculum

Python • GDSFactory • KLayout • Lumerical • PDKs • Compact Models • Verification • Tapeout •
Test

Canonical project roadmap — revision 2026-10-02

Goal: build an industry-grade, portfolio-quality silicon-photonics automation platform that connects design intent, parameterized layout, EM simulation, compact models, circuit assembly, verification, tapeout preparation, and test correlation.

How to use this curriculum in every project chat

1. Upload this PDF at the start of each chapter chat and treat it as the canonical curriculum.
2. Continue from the exact chapter/step reached previously; do not silently replace the roadmap with a different course structure.
3. For every meaningful concept: explain it, show one guided example, give a small assignment, review the result, then increase difficulty.
4. Assignments may introduce a small amount of new syntax, but the new syntax must be explained before or with the task.
5. Each chapter ends with a mini-project, file cleanup, README/documentation update, and a Git checkpoint.
6. Prefer industry-oriented photonics tasks over isolated Python exercises. Teach Python, software engineering, simulation, and layout in context.
7. As chapters advance, reduce hand-holding: by Chapters 5–8, assignments should increasingly resemble engineering specifications.
8. Keep generated layouts hierarchical and parameterized unless flattening is intentionally part of the lesson.

Why the roadmap was revised

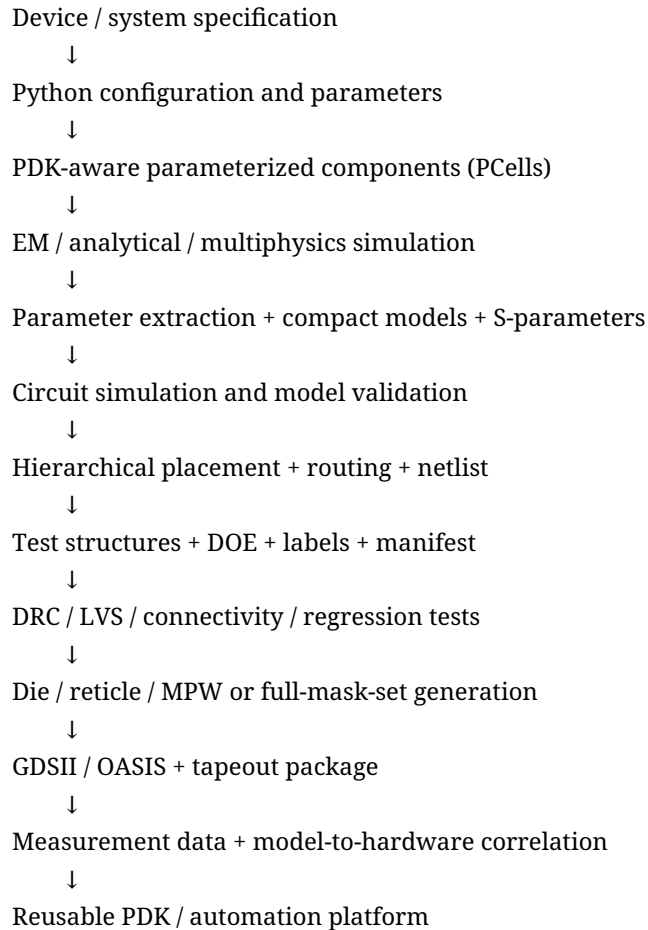
The original 8-chapter roadmap already covered GDSFactory fundamentals, reusable components, routing, silicon-photonic devices, simulation, test-chip generation, DRC/LVS, and a final automation capstone. This revision preserves that structure but strengthens areas repeatedly emphasized in current advanced photonics roles at IonQ, Xanadu, and Lightmatter:

- Internal PDK and PCell development, not merely PDK usage.
- Synchronization of physical layout, logical netlists, behavioral/compact models, and extracted parameters.
- Automated EM-to-model pipelines and model-to-hardware correlation.
- Regression testing, pytest, CI/CD, deterministic cell generation, and software-quality practices.
- GDSII and OASIS, MPW and dedicated/full-mask-set generation, reticle/die hierarchy, and tapeout checklists.
- Fabrication-aware layout: process bias, lithographic rounding, non-uniformity, density/fill concepts, and manufacturability.
- High-precision test structures: cutback, spirals, resonators, insertion-loss and propagation-loss extraction, and design-of-experiments.
- S-parameters, compact models, circuit simulation, netlist extraction, and circuit-level validation.
- Optional second EM backend and one non-silicon platform exercise to demonstrate solver/platform portability.

Industry target	What the roadmap must demonstrate
IonQ-style integrated photonics simulation/layout	Parameterized component libraries, PDK development, automated layout, full-reticle/tapeout ownership, DRC/LVS, Python layout, EM simulation, cross-platform photonics.
Xanadu-style photonics layout / integration	Expert scripted mask layout, GDSII/OAS, MPW/full mask sets,

	passive components, automated routing, ultra-low-loss test structures, fabrication-aware geometry, DRC/LVS, Git.
Lightmatter-style PDK / compact modeling	Python PCells, compact models, EM parameter extraction, layout-netlist-model synchronization, netlist extraction, circuit simulation, model-hardware correlation, pytest and CI/CD.

End-to-end capability target



Eight-chapter program overview

Chapter	Focus	Major finish artifact
01	Python & GDSFactory Fundamentals	Hierarchical two-channel mini-PIC + clean Git checkpoint
02	Components, Ports, PCells & Mini-PDK Foundations	Reusable component/PCell library + mini-PDK skeleton + tests
03	Routing, Connectivity & PIC Circuit Assembly	Parameterized multi-channel circuit + netlist/connectivity checks

04	Real Silicon-Photonics Devices & Test Structures	MZI/ring/coupler/Bragg library + loss/test structures
05	EM Simulation, Compact Models & Automated Sweeps	EM-to-compact-model automation pipeline + circuit validation
06	Automated Photonic Test Chip, DOE & Reticle Planning	Test-chip GDS/OAS + manifest + DOE + reticle/floorplan package
07	Physical Verification, Fabrication Effects & Tapeout	DRC/LVS-clean tapeout candidate + fabrication checklist/report
08	Industry-Style Automation Platform & Portfolio Capstone	Config-driven chip generator with CI/tests/models/reports

Chapter 01 — Python & GDSFactory Fundamentals

Purpose: Build the mental model for programmatic photonic layout and learn enough Python to work productively in GDSFactory.

Chapter finish condition: A working hierarchical mini-PIC, clean repository, README, final GDS, and GitHub checkpoint.

Step	Concept / skill	Minimum evidence
1.1	Python/GDSFactory environment and virtual environment	Script runs in the correct interpreter.
1.2	Generic PDK activation	Active generic PDK queried successfully.
1.3	<code>gf.Component()</code>	Create an empty named cell.
1.4	Standard components	Straight and Euler bend generated.
1.5	<code>add_ref()</code>	Child component placed by reference.
1.6	Component vs reference	Two references to one component placed independently.
1.7	Optical ports	Inspect name, center, width, orientation, layer, type.
1.8	<code>.connect()</code>	Port-to-port chain assembled without manual coordinates.
1.9	<code>.move()</code>	Reference translated explicitly.
1.10	Automatic routing	Separated ports connected with <code>route_single()</code> .
1.11	Cross-sections	Inspect strip width/layer; use <code>cross_section</code> in routing.
1.12	Bend-radius constraints	Compare $R=5/10/20\ \mu\text{m}$ and explain optical tradeoff.
1.13	Parent-level ports	Expose child ports at parent level.
1.14	Parameterized Python functions	Generate multiple geometry variants from one function.
1.15	<code>@gf.cell</code>	Parameterized reusable GDSFactory cell.
1.16	Cell caching	Demonstrate same parameters reuse cached cell.
1.17	Hierarchical layout	Straight-bend-straight reusable child cell.
1.18	Top-level component	Two child devices placed in a parent PIC.
1.19	Bounding box	Interpret <code>xmin/ymin/xmax/ymax</code> physically.

1.20	GDS export	Write a valid GDS file.
1.21	KLayout inspection	Verify geometry, layers and hierarchy.

Chapter 01 mini-project

Build a hierarchical two-channel photonic layout from one reusable `@gf.cell` generator. Use different device parameters, place the two channels at different coordinates, expose four top-level ports, inspect the bounding box, and export the GDS.

Chapter 01 finish checklist

- Mini-project runs and is visible in KLayout.
- Chapter README and curriculum are saved.
- `.gitignore` excludes virtual environments, IDE metadata, caches, and temporary files.
- A clean commit is created and pushed to GitHub.

Chapter 02 — Components, Ports, PCells & Mini-PDK

Foundations

Purpose: Move from basic syntax to a reusable, PDK-aware photonic component library and establish the first software-quality practices.

Chapter finish condition: A tested reusable component library plus a mini-PDK skeleton, GDS/OAS component gallery, README, and Git checkpoint.

2.1 — Port semantics and robust interfaces

Learn: Port center, orientation, width, layer, type, naming conventions, compatibility, and how ports transform with references.

Practice / assignment: Inspect straights, bends, tapers, MMIs and couplers; write a port-report utility; diagnose intentional port mismatches.

Evidence / artifact: port_report.py and a short table of component interfaces.

2.2 — Reference transformations

Learn: Translation, rotation, mirroring, coordinate frames, and how transformed references expose transformed ports.

Practice / assignment: Place four instances of one cell at different rotations/mirrors without duplicating the cell definition.

Evidence / artifact: transforms_gallery.gds

2.3 — Cross-sections as technology abstractions

Learn: Core width, layer, offsets/sections, cladding intent, bend/routing implications, and why cross-sections belong to technology definitions.

Practice / assignment: Create 450/500/600 nm strip variants from a single generator and inspect layers/ports.

Evidence / artifact: cross_section_sweep.gds

2.4 — Tapers and transitions

Learn: Adiabatic transition intent, width conversion, length, port compatibility, and taper hierarchy.

Practice / assignment: Build and parameterize a $0.45 \rightarrow 1.0 \mu\text{m}$ taper; connect it between mismatched waveguides.

Evidence / artifact: taper_library.py + GDS example.

2.5 — Splitters, MMIs and directional couplers

Learn: Multi-port components, port naming, hierarchy, coupling regions, and design intent versus foundry-qualified cells.

Practice / assignment: Inspect and place Y-branch/MMI/directional-coupler variants; expose clean parent ports.

Evidence / artifact: splitter_coupler_gallery.gds

2.6 — Reusable custom PCells

Learn: Parameterized @gf.cell generators, deterministic naming, validation of input parameters, docstrings, and API design.

Practice / assignment: Create a reusable waveguide/taper/coupler component module with meaningful defaults and parameter checks.

Evidence / artifact: components/ package.

2.7 — Mini-PDK skeleton

Learn: PDK anatomy: layers, cross-sections, component library, design constraints, naming conventions and technology abstraction.

Practice / assignment: Build a project-level mini-PDK with WG/SLAB/METAL/TEXT-style layers (generic/mock), standard cross-sections and component factories.

Evidence / artifact: pdk/ package with layers.py, cross_sections.py, components.py.

2.8 — First automated tests

Learn: Why photonic layout code needs software tests; deterministic ports, valid dimensions, repeatable cell generation.

Practice / assignment: Add simple pytest-style tests: expected port count/names, positive dimensions, correct layer, deterministic cache behavior.

Evidence / artifact: tests/test_components.py

2.9 — GDSII and OASIS awareness

Learn: Role of GDSII vs OASIS in mask delivery; hierarchical data and file-size considerations.

Practice / assignment: Export the component gallery in GDS; if supported in the installed toolchain, also export OAS and compare hierarchy/file size.

Evidence / artifact: component_gallery.gds / component_gallery.oas

Chapter 02 mini-project — Reusable Photonic Building-Block Library

- At least: straight, Euler bend, taper, splitter/MMI, directional coupler, custom routed section.
- Every reusable device is parameterized, exposes meaningful ports, uses @gf.cell where appropriate, and follows project naming rules.
- All devices use the mini-PDK layer/cross-section abstractions rather than scattering raw layer tuples through the code.
- Create a gallery top cell and a small automated test suite.

Chapter 02 finish checklist

- components/ and pdk/ modules are organized.
- Component gallery exports successfully.
- Tests pass.
- README documents API and port conventions.
- Git checkpoint: “Add reusable photonic component library and mini-PDK foundations”.

Chapter 03 — Routing, Connectivity & PIC Circuit Assembly

Purpose: Learn to assemble larger circuits from reusable cells while keeping placement, routing, connectivity and hierarchy explicit.

Chapter finish condition: A parameterized multi-channel circuit with automatic placement/routing, extracted connectivity representation, and regression checks.

3.1 — Placement versus routing

Learn: Separate floorplanning from interconnect generation; understand why good placement reduces routing loss and complexity.

Practice / assignment: Create the same two-device circuit with good and poor floorplans and compare route complexity.

Evidence / artifact: placement_vs_routing.gds

3.2 — Single optical routing in depth

Learn: route_single parameters, bend radius, cross-section, port orientations, route length and geometry constraints.

Practice / assignment: Route multiple separated port pairs and inspect route length/shape.

Evidence / artifact: routing_cases.gds

3.3 — Route bundles and channel arrays

Learn: Bundle routing, ordering, pitch, fanout, and avoiding crossings.

Practice / assignment: Route 4–8 input/output channels using a bundle strategy.

Evidence / artifact: bundle_routing.gds

3.4 — Routing conflicts and floorplan repair

Learn: Crossings, insufficient spacing, impossible bends, orientation conflicts and obstacle-aware reasoning.

Practice / assignment: Intentionally create a bad floorplan, document failure mode, then repair placement.

Evidence / artifact: routing_debug_notes.md

3.5 — Path-length accounting and matching

Learn: Geometric path length, optical path intent, ΔL , phase-sensitive circuits, and when matched routing matters.

Practice / assignment: Create two routes with controlled length difference and report lengths.

Evidence / artifact: path_length_control.gds

3.6 — Circuit hierarchy and sub-blocks

Learn: Input blocks, device blocks, output blocks, reusable circuit cells and top-level abstraction.

Practice / assignment: Build a hierarchical circuit from three reusable sub-blocks.

Evidence / artifact: hierarchical_circuit.gds

3.7 — Automated placement with loops and design rules

Learn: Pitch, grids, parameter arrays, loops, dictionaries/configuration and collision awareness.

Practice / assignment: Place a row/matrix of devices from a parameter list automatically.

Evidence / artifact: device_array.py

3.8 — Connectivity graphs and netlist thinking

Learn: Physical ports versus logical connectivity, component-instance graph, and why LVS/netlist extraction matters.

Practice / assignment: Create a simple Python connectivity representation from the intended circuit and compare it with placed references/ports.

Evidence / artifact: connectivity_manifest.json or .csv

3.9 — Routing regression tests

Learn: Connectivity and geometry should not silently change when code is refactored.

Practice / assignment: Test number of instances, expected top ports, route count, and device count.

Evidence / artifact: tests/test_circuits.py

Chapter 03 mini-project — Multi-Channel Routed PIC

- Generate a multi-channel PIC from a parameter list.
- Automatically place devices at a defined pitch.
- Route all channels without unintended crossings.
- Expose top-level I/O ports and generate a connectivity manifest.
- Add simple regression tests for device and route counts.

Chapter 03 finish checklist

- Parameterized circuit generator works.
- Connectivity representation agrees with design intent.
- Routing is clean in KLayout.
- Tests pass.
- Git checkpoint completed.

Chapter 04 — Real Silicon-Photonics Devices & Test Structures

Purpose: Build recognizable passive PIC devices, connect layout parameters to photonic physics, and add fabrication/test structures needed by real engineering teams.

Chapter finish condition: A documented device library plus high-precision loss/test structures and automated parameter arrays.

4.1 — Mach-Zehnder interferometer (MZI)

Learn: Splitter/combiner topology, arm lengths, ΔL , phase, FSR intuition, bend and routing implications.

Practice / assignment: Create balanced and unbalanced MZIs with parameterized ΔL and clean ports.

Evidence / artifact: mzi.py + gallery.

4.2 — Ring and racetrack resonators

Learn: Radius, coupling gap, coupling region, resonance, FSR, Q and loss sensitivity.

Practice / assignment: Generate ring/racetrack variants and a radius/gap sweep.

Evidence / artifact: rings.py + ring_sweep.gds

4.3 — Directional couplers

Learn: Gap, interaction length, supermodes/coupling intuition and wavelength dependence.

Practice / assignment: Create a parameterized directional coupler and sweep gap/length geometrically.

Evidence / artifact: directional_coupler.py

4.4 — MMIs and splitters

Learn: 1×2 / 2×2 multi-mode interference blocks, port conventions and use as circuit primitives.

Practice / assignment: Build an MMI-based splitter/combiner test cell.

Evidence / artifact: mmi_tests.gds

4.5 — Bragg gratings

Learn: Period, duty cycle, corrugation, number of periods, apodization concept and reflection filtering.

Practice / assignment: Generate hundreds of periods from parameters; build at least one chirped/apodized conceptual variant.

Evidence / artifact: bragg.py

4.6 — Spirals and cutback structures

Learn: Propagation-loss extraction, path-length scaling, footprint efficiency and bend contribution.

Practice / assignment: Generate straight/cutback/spiral variants with known path lengths.

Evidence / artifact: loss_test_structures.py

4.7 — High-precision insertion-loss structures

Learn: Why quantum/ultra-low-loss platforms need extremely sensitive differential structures; reference de-embedding and repeated-element strategies.

Practice / assignment: Design a matched reference/device pair where the difference isolates one component's excess loss.

Evidence / artifact: insertion_loss_test.gds

4.8 — Loss-origin identification concepts

Learn: Separate propagation, bend, coupling, scattering and absorption contributions through structured experiments.

Practice / assignment: Propose a small DOE using straight, bend-count, width and resonator structures to separate loss mechanisms.

Evidence / artifact: loss_doe.md

4.9 — Device arrays and naming

Learn: Automated parameter sweeps, deterministic labels, device IDs and experiment traceability.

Practice / assignment: Generate 2D arrays of ring/MZI/Bragg variants with encoded labels.

Evidence / artifact: device_arrays.gds

Chapter 04 mini-project — Silicon-Photonics Device & Test-Structure Library

- MZI, ring/racetrack, directional coupler, MMI/splitter, Bragg grating.
- At least two loss-characterization structures (e.g. cutback/spiral and differential insertion-loss structure).
- Each device has parameters, ports, short physical explanation, GDS example and unique naming.
- Arrays generated automatically from parameter sets.

Chapter 04 finish checklist

- Device library documented.
- Physics-to-parameter notes written.
- Test structures included.
- Gallery and parameter sweeps visible in KLayout.
- Git checkpoint completed.

Chapter 05 — EM Simulation, Compact Models & Automated Parameter Sweeps

Purpose: Connect geometry to electromagnetic behavior, automate parameter extraction, and package results into reusable compact models for circuit simulation.

Chapter finish condition: An EM-to-compact-model pipeline with automated sweeps, S-parameters/fit parameters, circuit-level validation, regression tests and stored datasets.

5.1 — Waveguide mode analysis

Learn: FDE/eigenmode basics, mode profiles, n_{eff} , n_g , confinement, loss and solver convergence.

Practice / assignment: Sweep waveguide width and wavelength in Lumerical MODE/FDE and store results.

Evidence / artifact: results/waveguide_modes.csv + plots.

5.2 — Single-mode and polarization analysis

Learn: TE/TM mode ordering, cutoff, higher-order modes and width/thickness dependence.

Practice / assignment: Determine a practical single-mode range for a chosen platform.

Evidence / artifact: single_mode_map.png

5.3 — Group index and dispersion

Learn: n_g , $dn_{\text{eff}}/d\lambda$, dispersion, MZI/ring FSR links.

Practice / assignment: Compute n_g numerically from $n_{\text{eff}}(\lambda)$ and compare with solver output if available.

Evidence / artifact: dispersion_analysis.py

5.4 — Bend simulation and loss

Learn: Radius-dependent radiation/mode-conversion loss, solver boundaries and convergence.

Practice / assignment: Sweep bend radius and compare layout compactness with simulated penalty.

Evidence / artifact: bend_loss.csv

5.5 — Directional-coupler EM model

Learn: Gap/length/wavelength dependence, supermode interpretation and coupling ratio.

Practice / assignment: Automate a gap $\times$ length sweep and identify a target ~50/50 design.

Evidence / artifact: coupler_sweep.csv

5.6 — Ring analytical/compact model

Learn: Transfer function, round-trip loss, phase, coupling, resonance, Q, extinction and FSR.

Practice / assignment: Implement a Python ring model and fit it to simulated parameters.

Evidence / artifact: models/ring_model.py

5.7 — Bragg model and simulation

Learn: TMM/coupled-mode intuition, spectrum versus period/corrugation/N.

Practice / assignment: Implement a simple analytical/TMM model and compare a selected case with EM simulation.

Evidence / artifact: models/bragg_model.py

5.8 — Lumerical Python automation

Learn: Programmatic setup, run, data extraction, file management and reproducibility.

Practice / assignment: Drive at least one solver sweep from Python rather than manual GUI repetition.

Evidence / artifact: simulation/automation scripts.

5.9 — Second solver/backend portability

Learn: Why teams value solver-independent automation; compare APIs/outputs rather than mastering every package.

Practice / assignment: Reproduce one small device or waveguide calculation using a second backend (Tidy3D, Meep, or COMSOL if available).

Evidence / artifact: backend_comparison.md

5.10 — S-parameters and component models

Learn: Port-based scattering matrices, reciprocity/passivity intuition, wavelength-dependent interpolation.

Practice / assignment: Export or synthesize S-parameters for a passive component and load them into a Python model.

Evidence / artifact: models/sparams/

5.11 — Circuit-level simulation

Learn: How compact models compose; use Python-native circuit tools (e.g. SAX/Simphony/Photontorch concepts) or Lumerical INTERCONNECT.

Practice / assignment: Build a simple MZI or ring circuit model from component models and compare with analytical expectations.

Evidence / artifact: circuit_simulation.py

5.12 — Automated parameter extraction

Learn: Fit compact-model parameters from EM outputs instead of hand-entering constants.

Practice / assignment: Create a script that ingests sweep data and emits a model parameter file.

Evidence / artifact: model_params.json

5.13 — Model-to-hardware correlation framework

Learn: Measurement ingestion, least-squares/statistical fitting, residuals, parameter updates and uncertainty.

Practice / assignment: Use synthetic or available measured data to fit at least one model and report residual error.

Evidence / artifact: correlation_report.ipynb/.py

5.14 — Data management and reproducibility

Learn: Folder structure, CSV/NPZ/HDF5 choices, metadata, simulation settings and run IDs.

Practice / assignment: Standardize results/ and metadata for every sweep.

Evidence / artifact: results/README.md

5.15 — Optimization and DOE

Learn: Grid sweeps, Latin-hypercube/design-of-experiments concepts, objective functions and constraints.

Practice / assignment: Optimize a coupler or taper against a target using automated simulation/model evaluation.

Evidence / artifact: optimization_summary.md

5.16 — Regression tests and CI-ready simulation checks

Learn: Fast unit tests versus expensive solver tests; golden data and tolerances.

Practice / assignment: Add lightweight tests for compact models and a small regression dataset.

Evidence / artifact: tests/test_models.py

Chapter 05 mini-project — EM-to-Compact-Model Automation Pipeline

- Choose one device (directional coupler or ring recommended).
- Parameter sweep is driven from Python.
- EM outputs are stored with metadata.
- Parameters/S-parameters are extracted automatically.
- A compact model is built and validated at circuit level.
- A fit/correlation step compares model against synthetic or measured data.
- Regression tests protect model behavior.

Chapter 05 finish checklist

- One-click/repeatable simulation pipeline.
- Saved datasets and plots.
- Compact model API documented.
- Circuit-level comparison completed.
- Tests pass and Git checkpoint completed.

Chapter 06 — Automated Photonic Test Chip, DOE & Reticle Planning

Purpose: Turn the component/device library into an experimentally useful chip with parameter sweeps, labels, manifests, references, I/O, floorplanning and reticle-aware organization.

Chapter finish condition: A complete automated test-chip GDS/OAS package with manifest, DOE table, floorplan/reticle plan, screenshots and documentation.

6.1 — Experimental matrix / DOE definition

Learn: Parameter combinations, controls, references, repeats and why not every combination is equally useful.

Practice / assignment: Define a structured DOE for ring radius/gap, MZI ΔL , Bragg period and waveguide width.

Evidence / artifact: configs/test_chip.yaml + DOE table.

6.2 — Waveguide/process characterization structures

Learn: Straight references, cutbacks, spirals, bend-count structures and width/process splits.

Practice / assignment: Place multiple loss structures with traceable IDs.

Evidence / artifact: test_structures block.

6.3 — Device arrays

Learn: Automated ring/MZI/Bragg/coupler array generation from configuration.

Practice / assignment: Generate arrays from the DOE without copy-pasted code.

Evidence / artifact: device_matrix.gds

6.4 — Optical I/O architecture

Learn: Grating versus edge couplers, fiber-array pitch, consistent orientation and testability.

Practice / assignment: Create an I/O block abstraction and attach it to selected test devices.

Evidence / artifact: io_block.py

6.5 — Labels and metadata

Learn: Human-readable labels, machine-readable IDs and parameter encoding.

Practice / assignment: Every test cell receives a unique device ID tied to a manifest.

Evidence / artifact: labels + manifest.

6.6 — Automated device manifest

Learn: Device ID, type, parameters, x/y, ports, test group and notes.

Practice / assignment: Generate CSV/JSON manifest directly from layout generation.

Evidence / artifact: device_manifest.csv

6.7 — Floorplanning

Learn: Die dimensions, margins, scribe streets, I/O edge, routing regions, reserved areas and keep-outs.

Practice / assignment: Define a die floorplan and place functional blocks by region.

Evidence / artifact: floorplan.gds + floorplan diagram.

6.8 — Reticle and MPW concepts

Learn: Die versus reticle hierarchy, MPW placement, dedicated mask sets, alignment and data handoff.

Practice / assignment: Build a mock reticle top cell containing one or more die references and frame/annotation cells.

Evidence / artifact: reticle_top.gds

6.9 — Alignment and process-control structures

Learn: Alignment marks, verniers/process monitors, measurement references and why they belong in a tapeout.

Practice / assignment: Add generic alignment and process-monitor cells.

Evidence / artifact: alignment/process cells.

6.10 — Density/fill awareness

Learn: Why foundries impose density/fill requirements; do not invent foundry fill rules without a real PDK.

Practice / assignment: Create a placeholder/fill region abstraction and document that real implementation is PDK/foundry-specific.

Evidence / artifact: fabrication_notes.md

6.11 — Automated chip generation

Learn: Configuration-driven chip build from device library + DOE + floorplan rules.

Practice / assignment: Implement generate_chip(config) that returns top-level chip and manifest.

Evidence / artifact: chip/generator.py

6.12 — GDSII/OAS export and release package

Learn: Multiple output formats, hierarchy preservation, deterministic release naming and artifact versioning.

Practice / assignment: Export release candidate and generate a release folder.

Evidence / artifact: releases/chip_vX/

Chapter 06 final project — Automated Silicon-Photonics Test Chip

- Waveguide loss structures, MZIs, rings/racetracks, Braggs, couplers/MMIs.
- Parameter sweeps generated from configuration.
- Optical I/O, labels, alignment/process-control structures.
- Device manifest and DOE table.
- Die floorplan plus mock reticle hierarchy.

- GDSII and, where supported, OASIS export.
- KLayout screenshots and architecture diagram.

Chapter 06 finish checklist

- Chip generator is deterministic and configuration-driven.
- Manifest matches layout population.
- No manual duplication of devices.
- Release folder contains GDS/OAS, config, manifest, plots/screenshots and README.
- Git checkpoint completed.

Chapter 07 — Physical Verification, Fabrication Effects & Tapeout

Purpose: Raise the standard from “looks correct” to “verifiable, manufacturable and release-ready,” while learning the limitations of a mock/generic PDK versus a real foundry PDK.

Chapter finish condition: A DRC/LVS/connectivity-checked tapeout candidate with fabrication-effect studies, hierarchy hygiene, checklist and release report.

7.1 — Real PDK anatomy

Learn: Layer maps, cross-sections, qualified cells, design rules, model libraries, process corners, extraction and documentation.

Practice / assignment: Map the project mini-PDK concepts to what a foundry PDK would provide; clearly mark mock versus real rules.

Evidence / artifact: pdk_gap_analysis.md

7.2 — Design-rule checking (DRC)

Learn: Minimum width/spacing/enclosure concepts, forbidden geometries, rule decks and why homemade checks are not equivalent to foundry signoff.

Practice / assignment: Create simple project-level checks and run KLayout DRC examples on intentionally bad structures.

Evidence / artifact: drc/ rules + report.

7.3 — Connectivity and LVS concepts

Learn: Design intent/netlist versus layout-extracted connectivity, port graph and hierarchy-aware verification.

Practice / assignment: Compare intended connectivity graph against generated layout/netlist representation.

Evidence / artifact: lvs/connectivity report.

7.4 — Netlist extraction

Learn: Instance/port graph extraction, hierarchical names and circuit representation.

Practice / assignment: Export a netlist-like representation for a circuit and verify expected connections.

Evidence / artifact: netlists/*.json or supported format.

7.5 — Fabrication bias and lithographic rounding

Learn: Etch/lithography bias, corner rounding, minimum radius and how “drawn” differs from “fabricated”.

Practice / assignment: Apply a controlled geometry bias/rounding study to selected structures; compare dimensions/expected impact conceptually.

Evidence / artifact: fabrication_bias_study.gds

7.6 — Process variation / corner analysis

Learn: Width/thickness/index/gap variation and sensitivity; worst-case/corner thinking.

Practice / assignment: Run compact-model or EM sweeps at nominal and perturbed geometry corners.

Evidence / artifact: corner_analysis.csv

7.7 — OPC and mask-data-preparation concepts

Learn: What OPC/MDP/fill/fracturing are and why they are usually foundry/mask-house responsibilities.

Practice / assignment: Document the boundary between designer-side pre-bias and foundry OPC; do not fabricate unsupported rules.

Evidence / artifact: mdp_notes.md

7.8 — Cell naming, hierarchy and duplicate-cell hygiene

Learn: Stable names, references, flattening risks, library collisions and release reproducibility.

Practice / assignment: Audit generated hierarchy and eliminate accidental duplicate/ambiguous cells.

Evidence / artifact: hierarchy_report.txt

7.9 — Full-mask-set / reticle release checks

Learn: Top cell, die/reticle hierarchy, layer presence, bounds, labels, scribe area, required manifests and checksum/version discipline.

Practice / assignment: Run a preflight script that checks release folder completeness and top-cell sanity.

Evidence / artifact: preflight.py + report.

7.10 — Non-silicon platform portability exercise

Learn: Adapt abstractions to a second platform (e.g. generic TFLN/diamond/SiN) without pretending to possess a proprietary PDK.

Practice / assignment: Define a second mock cross-section/layer set and port-compatible component subset; note material/platform implications.

Evidence / artifact: pdk_alt_platform/

7.11 — Tapeout review process

Learn: Peer review, signoff checklist, known waivers, foundry questions, release notes and archival artifacts.

Practice / assignment: Perform a structured design review of the Chapter 06 chip.

Evidence / artifact: TAPEOUT_CHECKLIST.md

Chapter 07 mini-project — Fabrication-Oriented Signoff of the Test Chip

- Run project-level DRC and connectivity checks.
- Generate/verify a netlist-like connectivity representation.
- Perform nominal + geometry-corner sensitivity analysis.
- Audit hierarchy and top-cell release structure.
- Prepare tapeout checklist and release notes.

- Export final GDS/OAS candidate and archive release metadata.

Chapter 07 finish checklist

- All project-level checks pass or waivers are documented.
- Mock rules are clearly distinguished from foundry signoff.
- Tapeout/release checklist complete.
- Hierarchy is clean and reproducible.
- Git checkpoint completed.

Chapter 08 — Industry-Style Automation Platform & Portfolio Capstone

Purpose: Integrate layout, PDK abstractions, EM extraction, compact models, circuit simulation, DOE, verification, release automation and documentation into one professional project.

Chapter finish condition: A configuration-driven photonic design-automation platform with tests/CI, compact models, chip generation, verification, release reports and a polished portfolio repository.

8.1 — Repository and package architecture

Learn: Professional Python package layout, modules, configs, examples, tests, docs and releases.

Practice / assignment: Reorganize into src/ package structure with clean imports.

Evidence / artifact: src/pic_automation/...

```
PIC-silicon-photonics-automation/  
├─ README.md  
├─ pyproject.toml  
├─ configs/  
├─ src/pic_automation/  
│   ├── pdk/  
│   ├── components/  
│   ├── circuits/  
│   ├── routing/  
│   ├── simulation/  
│   ├── models/  
│   ├── verification/  
│   └─ chip/  
├─ tests/  
├─ examples/  
├─ results/  
├─ releases/  
└─ docs/
```

8.2 — Configuration-driven design

Learn: YAML/JSON parameters, schema validation, defaults and reproducibility.

Practice / assignment: Generate a device/chip without editing source code.

Evidence / artifact: configs/test_chip.yaml

8.3 — PDK/component integration

Learn: Single source of truth for layers, cross-sections, PCells and design constraints.

Practice / assignment: Route every component through the project PDK API.

Evidence / artifact: pdk package finalization.

8.4 — Layout-netlist-model synchronization

Learn: Physical cell, logical ports/netlist and compact model must represent the same component and parameter set.

Practice / assignment: Define a component registry that links layout generator + model + metadata.

Evidence / artifact: component_registry.py

8.5 — Simulation integration

Learn: Automated solver dispatch, caching, run IDs, extraction and model packaging.

Practice / assignment: Generate/update model data for selected components from simulation outputs.

Evidence / artifact: simulation pipeline.

8.6 — Circuit simulation and design selection

Learn: Use compact models to evaluate higher-level performance and choose design candidates.

Practice / assignment: Select a coupler/ring/MZI candidate against a target specification.

Evidence / artifact: design_selection_report.md

8.7 — Automated chip assembly

Learn: DOE, placement, routing, I/O, labels, manifests and reticle hierarchy from configuration.

Practice / assignment: One command generates the full chip.

Evidence / artifact: generate_chip.py

8.8 — Verification automation

Learn: DRC-like project checks, connectivity/netlist checks, parameter sanity, release preflight.

Practice / assignment: Run verification automatically as part of chip generation or release.

Evidence / artifact: verify_release.py

8.9 — pytest and CI/CD

Learn: Unit tests, regression tests, fast/slow markers, continuous integration workflow and artifact checks.

Practice / assignment: Set up CI to run fast tests on every push and optionally selected regression checks.

Evidence / artifact: .github/workflows/ci.yml or equivalent.

8.10 — Model-hardware correlation demo

Learn: Close the loop from design to measurement; fit/update models from data.

Practice / assignment: Use measured data if available; otherwise a clearly labeled synthetic dataset to demonstrate the workflow.

Evidence / artifact: correlation pipeline.

8.11 — Automated reporting

Learn: Generate device tables, model parameters, sweep plots, verification status, release metadata and screenshots.

Practice / assignment: Create a reproducible release report from generated artifacts.

Evidence / artifact: reports/release_report.pdf/html

8.12 — Portfolio documentation

Learn: Explain problem, architecture, supported devices, simulation flow, verification, reproduction steps and limitations.

Practice / assignment: Write a recruiter/engineer-friendly README with architecture diagram and results.

Evidence / artifact: Final README + docs.

8.13 — Final engineering challenge

Learn: Operate from a specification rather than step-by-step instructions.

Practice / assignment: Design a test chip that characterizes waveguide width, resonator/coupler parameters and loss while fitting a die/reticle area and passing project checks.

Evidence / artifact: Final capstone release.

Final capstone command target

```
python generate_chip.py --config configs/test_chip.yaml
```

The target is that this command (or an equivalent package entry point) generates the device population, layout, manifest, verification results and release artifacts deterministically.

Chapter 08 finish checklist

- Python/GDSFactory component and circuit library.
- Mini-PDK abstraction and component registry.
- MZI, ring/racetrack, Bragg, coupler/MMI and loss-test structures.
- Automated EM sweeps and parameter extraction.
- Compact/S-parameter models and circuit-level simulation.
- Layout-netlist-model synchronization.
- Configuration-driven test-chip/reticle generation.
- DRC/connectivity/preflight automation.
- pytest + CI/CD.
- Final GDS/OAS + manifest + reports + plots.
- Professional README and clean Git history.

Assignment difficulty progression

Stage	Typical assignment style
Ch. 1–2	“Create this component, inspect these ports, parameterize these dimensions.”
Ch. 3	“Place and route this multi-channel circuit while satisfying orientation and spacing constraints.”
Ch. 4	“Build a device/test structure from physical specifications and explain the physics behind each parameter.”
Ch. 5	“Automate this simulation sweep, extract a model, validate it, and quantify error.”
Ch. 6	“Design a DOE and generate an experimentally useful chip with labels, manifest and floorplan.”
Ch. 7	“Prepare this design for release: verification, corners, hierarchy, signoff checklist.”
Ch. 8	“Here is the system specification. Decide the architecture and deliver a reproducible release.”

Portfolio evidence expected by the end

- Readable Python package with reusable PCells and mini-PDK abstractions.
- GDS/OAS examples and KLayout screenshots showing hierarchy and routing.
- Simulation notebooks/scripts plus saved sweep data and plots.
- Compact-model and S-parameter examples.
- Circuit-level model validation.
- Loss/test structures and DOE methodology.
- Automated manifest and configuration-driven chip generation.
- DRC/connectivity/preflight outputs and tapeout checklist.
- Unit/regression tests and CI workflow.
- Final release package and professional README.

Approximate effort

Chapter	Approximate effort	Why
01	10–15 h	Foundation, environment, GDSFactory mental model.
02	20–30 h	Reusable PCells, mini-PDK, tests and component library.
03	20–30 h	Routing, connectivity, automated

		placement and circuits.
04	30–45 h	Real devices plus test/loss structures and physics.
05	55–80 h	EM automation, compact models, extraction, correlation, circuit simulation.
06	35–50 h	DOE, test chip, I/O, manifest, floorplan and reticle hierarchy.
07	30–45 h	Verification, variation, fabrication effects and tapeout discipline.
08	50–80 h	Integration, software architecture, CI, reporting and final capstone.

Total expected scale: roughly 250–375 hours depending on simulation depth, access to commercial tools/PDKs, and how far the project is taken toward measurement/tapeout realism. This is intentionally a serious portfolio program rather than a short tutorial.

Competency coverage against target roles

Competency	Primary chapter(s)	Evidence
Scripted photonic layout / GDSFactory	1–4, 6–8	PCell library, circuits, chip generator.
PDK development / support	2, 7, 8	Mini-PDK, component registry, PDK gap analysis.
DRC / LVS / connectivity	3, 7, 8	Connectivity graph, DRC rules, netlist checks, release verification.
GDSII / OAS / MPW / full-mask-set concepts	2, 6, 7	GDS/OAS export, die/reticle hierarchy, release package.
Passive components	4	Gratings, rings, racetracks, directional couplers, MMIs, MZIs.
Ultra-low-loss test structures	4, 6	Cutback, spiral, differential insertion-loss, DOE.
Fabrication effects	7	Bias, rounding, process corners, OPC/MDP concepts.
EM automation	5	Python-driven sweeps and data extraction.
Compact models / S-parameters	5, 8	Model library, parameter extraction and circuit validation.
Model-to-hardware correlation	5, 8	Fit/correlation workflow.

Netlist extraction / synchronization	3, 7, 8	Connectivity/netlist + component registry.
pytest / CI/CD / software rigor	2, 3, 5, 8	Tests and CI workflow.
Tapeout / release discipline	6, 7, 8	Reticle/floorplan, checklist, release candidate.
Cross-platform / non-silicon adaptability	5, 7	Second solver and second mock material platform exercise.

Source basis for this revision

This revision was grounded in the previous project curriculum and cross-checked against current role descriptions whose skill requirements motivated the new emphasis on PDK engineering, full-mask-set/tapeout workflows, compact modeling, test structures, regression testing and model-to-hardware correlation.

- IonQ — Lead Integrated Photonics Simulation and Layout Engineer (current 2026 posting; role emphasizes parameterized component libraries, PDK work, automated layout, simulation, physical verification and tapeout ownership).
- Xanadu — Photonics Design Engineer (Layout) (role emphasizes scripted photonic mask design, DRC/LVS, GDSII/OAS, MPW/full mask sets, passive components, ultra-low-loss test structures, fabrication effects and automated routing).
- Lightmatter — Staff Photonics Design Engineer - Compact Modeling / Photonics PDK Modeling Engineer (role emphasizes Python PCells, compact models, EM parameter extraction, layout/netlist/model synchronization, model-hardware correlation, regression tests, CI, netlist extraction and circuit simulation).
- Lukas Chrostowski & Michael Hochberg, Silicon Photonics Design — used as a background reference for the broader industry design flow: component modeling, physical layout, verification, PDKs, DRC/LVS, fabrication and testing.

Important boundary conditions

- A generic or self-created mini-PDK is educational; it is not a substitute for a confidential foundry PDK or foundry signoff decks.
- Project-level DRC/LVS exercises teach methodology but cannot be called foundry-clean unless run with the actual foundry rules and approved tools.
- Synthetic measurement data may be used to demonstrate model-correlation workflows when real fabricated data is unavailable, but it must be labeled synthetic.
- The curriculum aims to demonstrate strong transferable capability; it does not replace industrial tapeout history required by senior/lead job postings.

Canonical continuation rule

When this PDF is uploaded into a future chapter chat, use it as the authoritative roadmap. Identify the current chapter and step, preserve completed work, and continue with the next unfinished step using the explain → guided example → assignment → review → harder task → mini-project → cleanup → Git-checkpoint progression. Do not silently substitute a different curriculum.